

Tema 4. Diseño Orientado a Objetos.

De los Requisitos al diseño. Objetos Y Clases. Proceso del Diseño Orientado a Objetos. Contexto del Sistema y Modelos de Utilización. Diseño de la Arquitectura. Identificación de Objetos. Modelo de Diseño. Especificación de la Interfaz de Objetos. La documentación del Diseño y la importancia de la trazabilidad. Objetos Concurrentes. Evolución del diseño. Diseño de Software de Tiempo Real. Diseño de Interfaces de Usuario.

Análisis y Diseño orientado a objetos.

Se apoya en UML, un lenguaje de modelado unificado que permite representar, construir y documentar los elementos de un sistema de software orientado a objetos. UML proporciona una manera estructurada de visualizar y entender un sistema desde diferentes perspectivas, permitiendo a los desarrolladores fijarse en aspectos específicos del sistema a través de modelos detallados.

UML.

Es un lenguaje gráfico utilizado para visualizar, especificar, construir y documentar las partes de un sistema de software desde diferentes perspectivas. Proporciona una forma estándar de modelar sistemas de software que puede adaptarse a cualquier proceso de desarrollo, ciclo de vida y tecnología de implementación. Además, UML se puede aplicar en diversas áreas más allá del desarrollo de software, como la ingeniería de negocios y el modelado de procesos. UML consta de una variedad de notaciones que permiten describir tanto los aspectos estáticos como dinámicos de un sistema, incluyendo sus elementos y relaciones, así como su comportamiento a lo largo del tiempo.

Tipos de Diagramas.

Existen dos grupos principales de diagramas: los diagramas de Estructuras y los diagramas de Comportamiento.

- **Los Diagramas de Estructuras:** Ofrecen una vista estática del modelo y se centran en los elementos que deben existir en el sistema modelado.

Incluyen el Diagrama de Clases, Diagrama de Componentes, Diagrama de Objetos, Diagrama de Despliegue o Distribución, y Diagrama de Paquetes.

- **Los Diagramas de Comportamiento:** Ofrecen una vista dinámica del modelo y se centran en lo que debe suceder en el sistema. Incluyen el Diagrama de Actividad, Diagrama de Caso de Uso, y Diagrama de Estados.

- **Los Diagramas de Interacción:** Como el Diagrama de Secuencia y el Diagrama de Comunicación o Colaboración, son un subtipo de los diagramas de comportamiento que muestran cómo interactúan los elementos del sistema en un momento dado.

Del Análisis al Diseño.

En el proceso de desarrollo de software, el análisis se enfoca en describir el sistema desde el punto de vista de los actores, sin incluir detalles sobre la estructura interna del sistema. Por otro lado, el diseño del sistema implica la optimización de los objetivos de diseño derivados de los requisitos no funcionales, así como la definición de una arquitectura de software que incluya la descomposición en subsistemas y las dependencias entre ellos. El diseño detallado implica el refinamiento del diseño del sistema y la definición de las clases de la solución, incluyendo sus interfaces.

Objetos y Clases.

Una clase en programación orientada a objetos describe grupos de objetos que comparten propiedades (atributos) similares, comportamientos (operaciones) comunes, relaciones con otros objetos y semántica común. Cada objeto creado a partir de una clase sabe cuál es su clase, ya que es una instancia de la misma.

Cuestiones por Resolver.

Al abordar un proyecto de desarrollo de software, es importante resolver varias cuestiones clave:

1. Comprender y definir el contexto y las interacciones externas con el sistema.
2. Diseñar la arquitectura del sistema.
3. Identificar los objetos principales en el sistema.
4. Desarrollar modelos de diseño.
5. Especificar interfaces.

Recordar la naturaleza creativa del proceso, por lo tanto, será iterativo...no secuencial. (I. Sommerville).

Primeras fases del proceso de diseño.

1. **Modelos de subsistema:** Que muestran los agrupamientos lógicos de objetos en subsistemas coherentes. Se representan mediante diagramas de clases donde cada subsistema se muestra como un paquete con objetos contenidos. Estos modelos son estáticos y estructurales.
2. **Modelos de secuencia:** Que ilustran la secuencia de interacciones entre objetos. Se representan mediante diagramas de secuencia UML o diagramas de colaboración. Estos modelos son dinámicos.
3. **Modelos de máquina de estado:** Que muestran cómo los objetos individuales cambian su estado en respuesta a eventos. Se representan en UML mediante diagramas de estado. Estos modelos también son dinámicos.

Especificación de Interfaz.

Se enfoca en definir los detalles de cómo interactuarán los objetos o grupos de objetos. Esto implica especificar las firmas y la semántica de los servicios que ofrecerá cada objeto o grupo de objetos. En UML, las interfaces pueden ser especificadas utilizando la misma notación que se utiliza en un diagrama de clase, pero con el estereotipo UML «interface» en el nombre. Además, la semántica de la interfaz se define mediante el lenguaje de restricción de objeto (OCL).

Definición de una clase.

- Identificar y nombrar la clase.

- Identificar sus componentes.
- Identificar sus atributos.
- Identificar los errores.
- Identificar las conexiones funcionales (qué clases sirve/exige).
- Definir conexiones con superclase y subclases.
- Identificar propiedades especiales (persistencia, concurrencia).
- Probar la clase en un prototipo.

Identificar sus atributos.

El conjunto de atributos de una clase debe ser:

- Completo (contienen toda la información pertinente).
- General (se aplican a todos los objetos de la clase).
- Diferenciado (cada atributo representa un aspecto diferente de la clase).

Definir atributos.

- Tipos de atributos:
 - Atómicos predefinidos (entero, real, carácter, pixel...).
 - Atómico enumerativo (color, día de la semana...).
 - Colección.
 - Composición (referencias objetos).
- Valor del atributo:
 - Común a muchos objetos (variable de clase).
 - Propio de un objeto (variable de objeto).

Modelo de Objetos y DCC.

Se utiliza para representar la estructura y el comportamiento de todas las clases identificadas en el dominio del problema, junto con sus relaciones. El DCC es un

modelo semántico extendido que proporciona una visión detallada de cómo interactúan las clases en el sistema y cómo se relacionan entre sí.

Crear el diagrama de Clases de Diseño.

El proceso comienza identificando las clases que participarán en la solución de software. Esto se logra examinando los diferentes diagramas de interacción y listando las clases mencionadas en ellos. Además, los nombres de los métodos pueden ser identificados analizando los diagramas de interacción, lo que ayuda a completar la especificación de las clases en el DCD.

Operaciones y Métodos.

- Función o transformación que se aplica 'a' o 'por' los objetos en una clase.
- Tienen un objeto destino como argumento implícito (el objeto actual).
- **Polimorfismo:** la misma operación puede aplicarse a clases diferentes dentro de una jerarquía de herencia.
- **Método:** implementación de una operación para una clase. Las operaciones con métodos en varias clases deben compartir la misma signatura (tipos de sus argumentos y valor que devuelven).

Identificar Métodos.

- **Método:** algoritmo que utiliza y modifica los atributos de una clase. Un método es desencadenado por un mensaje.
- **Funcionalidad de la clase:** conjunto de sus métodos. El conjunto de métodos debe ser:
 - Completo (realizan toda la funcionalidad de la clase).
 - General (se aplican a todos los objetos de la clase).
 - Diferenciado (cada método debe ser simple y realizar una sola función).

Definir Métodos.

- Tipos de métodos:

- Modificador (asigna valor a un atributo).
- Selector (devuelve el valor de un atributo).
- Aplicable a la clase (constructor).
- Aplicable al objeto.
- Parámetros del método:
 - ¿Qué información necesita? (argumentos de entrada).
 - ¿Qué debe devolver? (resultado y argumentos salida).

Identificar Errores.

- ¿Qué puede salir mal durante la ejecución de un método?
- ¿Qué comprobaciones debe hacer cada método?
- ¿Cómo interceptar y corregir las condiciones de error?

Enlaces y Asociaciones.

- Enlace: conexión física o conceptual entre objetos.
- Asociación: grupo de enlaces con la misma estructura y semántica común.
- Sentido de una asociación:
 - Inherentemente son bidireccionales. Directa (forward) e inversa (inverse).

Asociaciones.

Las asociaciones pueden ser binarias, ternarias o de órdenes superiores. Los nombres de las asociaciones son opcionales en la notación (se escriben en cursiva).

Multiplicidad.

La multiplicidad en el contexto de la modelización de sistemas orientados a objetos especifica cuántos objetos de una clase pueden estar relacionados con uno o varios objetos de otra clase asociada. Se puede representar mediante un subconjunto de enteros no negativos (por ejemplo, 0..n) o mediante intervalos no conexos (por ejemplo, 1..4, 7). Es importante no preocuparse por ajustar la multiplicidad demasiado pronto en el proceso de modelado, ya que puede requerir revisión durante la etapa de diseño. En los Diagramas de Objetos, la multiplicidad se muestra mediante símbolos especiales en los extremos de las líneas que representan las asociaciones entre las clases.

Relaciones.

Las acciones son servicios que un objeto puede activar (actuando como agente) para consultar o modificar el estado de otro objeto.

Las relaciones estructurales que podemos modelar son la agregación (parte-de) y la herencia (es- un).

Agregación.

Se presenta la relación de Agregación entre clases, incluyendo información sobre cardinalidades (mínimas y máximas) que determinan cuántos objetos componentes forman parte de un objeto compuesto e inversamente, cuántos objetos compuestos pueden estar compuestos de un objeto en particular.

Herencia.

Se define gráficamente trazando una flecha desde la subclase hacia la superclase correspondiente. Esta flecha de especialización puede etiquetarse con una condición de especialización o con los eventos correspondientes de activación/cancelación de la especialización temporal (rol).

Herencia / Generalización.

Generalización: relación entre una clase (superclase) y una o más versiones refinadas de ella (subclases). Relación "es-un". Las subclases 'heredan' las

características, atributos y operaciones de su superclase. Una instancia de una subclase es una instancia de sus clases antecesoras o ascendientes.

Distinción entre generalización y herencia.

- Generalización: relación entre clases.
- Herencia: mecanismo para compartir características.

Recordar.

Sobre el diagrama de clases construido revisar:

- Relaciones y cardinalidades entre clases.
- Mantener coherencia con los demás diagramas que forman parte de la documentación del sistema.
- Respetar los criterios de Cohesión y Acoplamiento.
- Al implementar, primero inicio por las clases entidades (menos dependencias) y luego por las controladoras (lógica del negocio).

Tema 5. Patrones de Diseño.

Patrones de Diseño. Aplicación de Graps. Conceptos Fundamentales. Características principales de los distintos Patrones. Diseño de la realización del caso de uso con patrones de diseño. Adaptador. Factoría. Singleton. Estrategia. Composite. Fachada. Observador/Publicar/Suscribir. Modelos de Delegación de Eventos. Ventajas sobre su aplicación.

Introducción.

El diseño es un modelo del sistema realizado con una serie de principios y técnicas, permitiendo describir el sistema con suficiente detalle para ser implementado. Estos esquemas y estructuras reusables facilitan la reutilización ante problemas similares, llevando a arquitecturas más simples y comprensibles.

Hacer software no es fácil.

Diseñar software orientado a objetos es difícil y hacerlo reutilizable es aún más desafiante. Un software que pueda evolucionar debe ser reutilizable, al menos para futuras versiones.

Diseñar para el cambio.

El software está sujeto a cambios. Para anticiparse a los cambios en los requisitos, es necesario diseñar pensando en qué aspectos pueden cambiar. Los patrones de diseño están orientados a facilitar estos cambios.

Patrones.

Un patrón es una solución a un problema en un contexto particular que se repite y puede enseñarse para adaptarlo a variaciones específicas del problema. Los patrones facilitan la reutilización de diseños y arquitecturas software exitosos.

Motivación de los Patrones.

Los patrones de diseño son herramientas que capturan la experiencia y la hacen accesible a todos, mejorando la comunicación entre desarrolladores y facilitando la comprensión de sistemas. También sirven como base para manuales de ingeniería

de software y facilitan la reestructuración de sistemas, incluso si no fueron concebidos con patrones en mente. Además, apoyan la reutilización de arquitecturas y diseño, y están orientados al cambio, ayudando a anticipar y adaptarse a los cambios en los requisitos del software.

Clasificación.

Los patrones de diseño se clasifican en tres categorías principales:

- **Arquitecturales:** Proporcionan un paradigma fundamental para estructurar un sistema software.
- **De Diseño:** Describen el esquema básico para estructurar subsistemas y componentes.
- **Elementales (idioms):** Específicos de un lenguaje de programación, describen cómo implementar componentes particulares de un patrón.

Características.

Los patrones de diseño son descripciones de clases que colaboran entre sí para resolver ciertos problemas. Cada patrón tiene un nombre, un propósito, sinónimos, el problema al que se aplica, estructura, participantes, colaboraciones, implementación y relaciones con otros patrones. Estas características proporcionan una guía para su aplicación y comprensión en el diseño de software.

Ejemplos de patrones.

- **Patrones Arquitecturales:** Jerarquía de capas, tuberías y filtros, cliente/servidor, maestro-esclavo, control centralizado y distribuido.
- **Patrones de Diseño:** Proxies, factorías, adaptadores, composición, broker.
- **Patrones Elementales:** Modularidad, interfaces mínimas, encapsulación, objetos, acciones y eventos, concurrencia.

Ventajas.

Los patrones de diseño permiten anticiparse a nuevos requisitos y cambios, facilitando la evolución del sistema. Cada patrón permite modificar aspectos específicos de la estructura sin afectar otros, ofreciendo soluciones a problemas

comunes. Además, promueven la reutilización, extensibilidad y mantenimiento del software.

Patrones y Armazones (Frameworks).

Los patrones de diseño son descripciones abstractas y más pequeñas que los armazones (frameworks). Los armazones están implementados en un lenguaje de programación y pueden ser ejecutados y reutilizados directamente. Un armazón incorpora varios patrones y se aplica a un dominio de aplicación particular.

Características de los Armazones (Frameworks).

Un armazón ofrece un conjunto integrado de funcionalidad específica de un dominio y determina qué métodos se invocan en respuesta a eventos. Es una aplicación medio-acabada, reutilizando el código del cuerpo principal y permitiendo desarrollar aplicaciones completas mediante herencia e instanciación de componentes parametrizados del armazón.

Descripción de patrones de diseño.

Un patrón describe un problema recurrente en un contexto específico y ofrece una solución que puede reutilizarse múltiples veces. Facilita la reutilización de diseños y arquitecturas exitosas.

Cómo seleccionar un patrón de diseño.

Para seleccionar un patrón de diseño adecuado:

- Considerar cómo los patrones solucionan problemas de diseño.
- Buscar las intenciones de cada patrón.
- Estudiar la interrelación y propósito similar de los patrones.
- Examinar la causa del rediseño.
- Considerar qué debería ser variable en un diseño.

Cómo usar un patrón de diseño.

Los pasos para usar un patrón de diseño incluyen:

1. Leer el patrón para obtener una visión general.

2. Estudiar la estructura, participantes y colaboraciones.
3. Ver ejemplos codificados.
4. Elegir nombres significativos para los participantes.
5. Definir clases y operaciones específicas de la aplicación.
6. Implementar las operaciones necesarias para las responsabilidades y colaboraciones del patrón.

Patrones de creación.

Permiten que el sistema sea independiente de cómo se crean, componen o representan sus objetos. En Java, se utiliza el operador 'new' para crear un objeto.

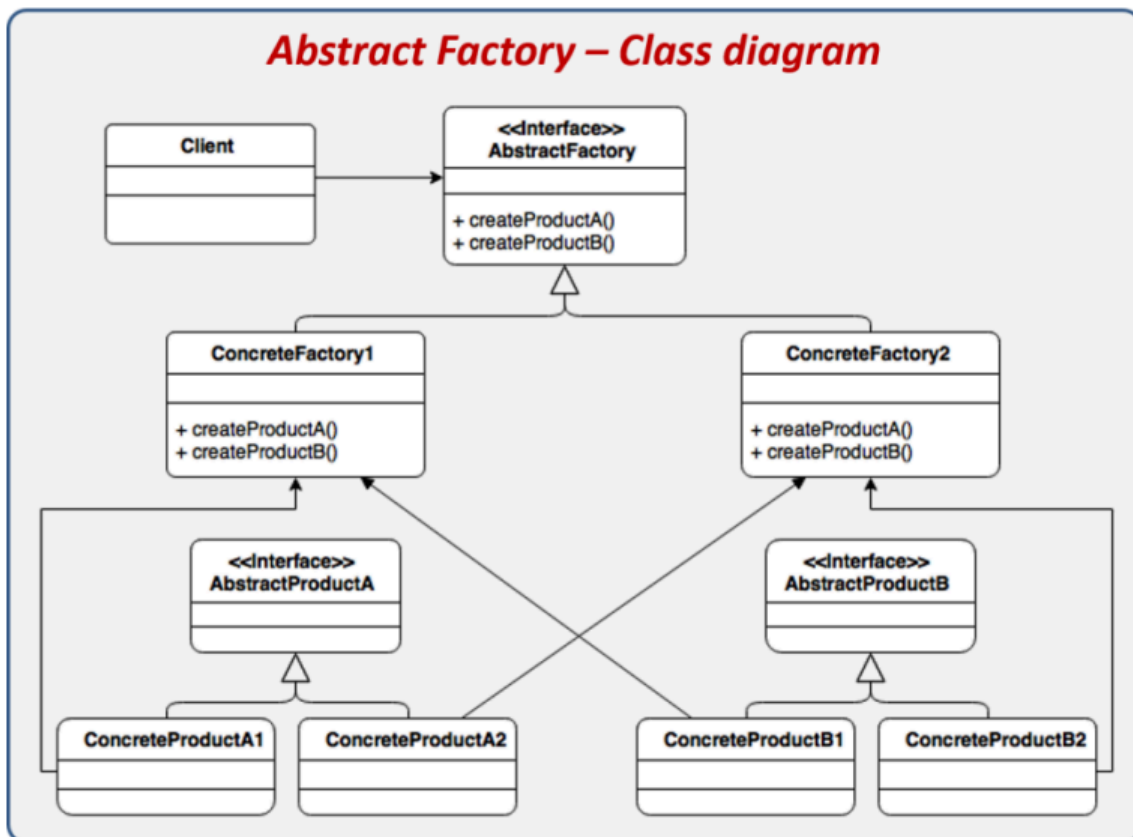
Tipos de patrones de creación.

Incluyen Método Fábrica Abstracta, Método Fábrica, Constructor, Prototipo, Singular cada uno con un propósito específico para mejorar la creación de objetos y la consistencia en el sistema.

1. Fábrica Abstracta (Abstract Factory)

- Descripción: Proporciona una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.
- Problema: Cuando el sistema debe ser independiente de cómo se crean, componen y representan sus productos.
- Solución: Define interfaces para cada tipo de producto y una fábrica que los produzca, permitiendo cambiar la familia de productos sin alterar el código cliente.
- Elementos:
 - AbstractFactory (interfaz).
 - ConcreteFactory.
 - AbstractProduct.
 - ConcreteProduct.
 - Cliente
- Ventajas:
 - Aísla la creación de productos concretos.

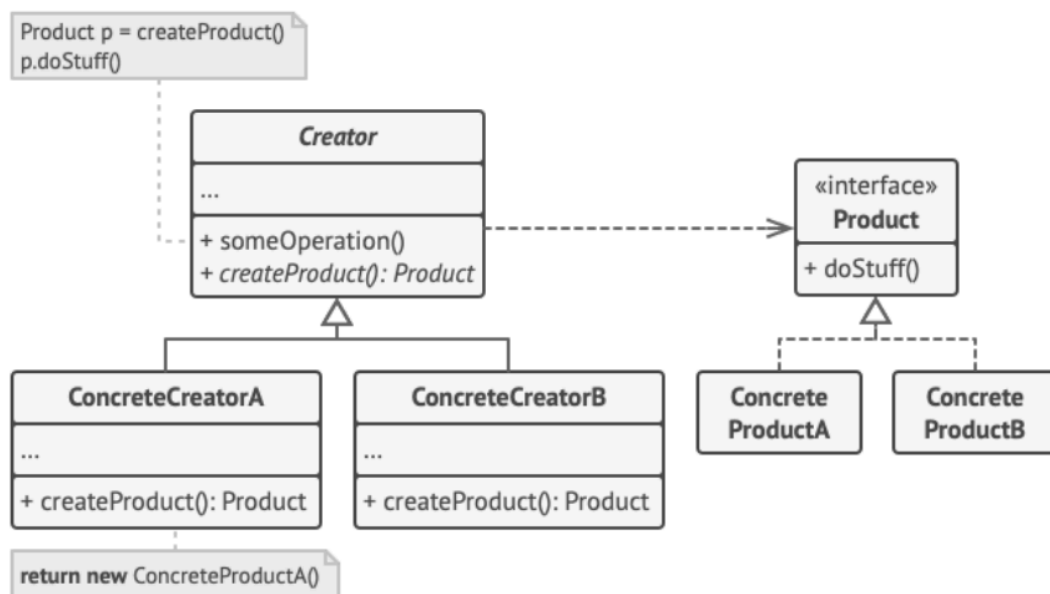
- Facilita el intercambio de familias de objetos.
- Promueve la consistencia entre objetos relacionados.
- Desventajas:
 - Aumenta la complejidad del código.
 - Dificultad para agregar nuevas familias de productos.
- Aplicaciones:
 - Sistemas con múltiples familias de productos (e.g., interfaces gráficas para diferentes SO).
 - Cuando se desea garantizar la compatibilidad entre objetos.
- Gráficamente:



2. Método Fábrica (Factory Method)

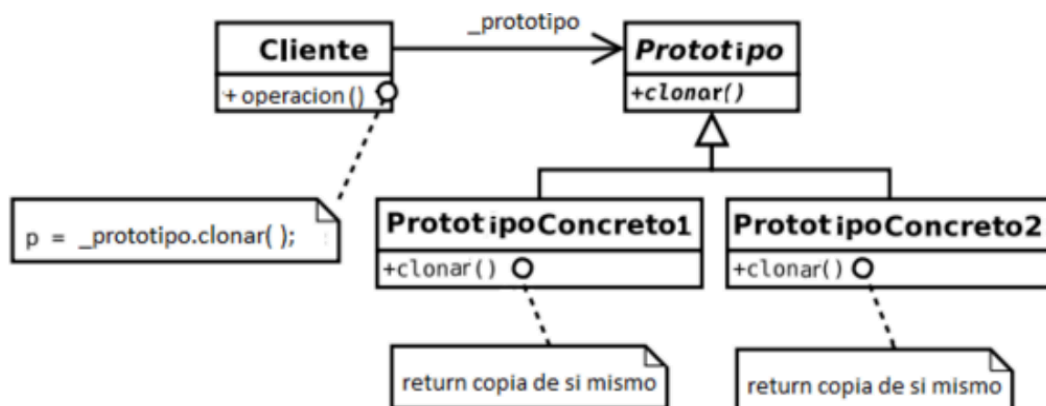
- Descripción: Define una interfaz para crear un objeto, pero permite a las subclases decidir qué clase instanciar. Delega la creación a subclases.

- Problema: Un sistema debe delegar la creación de objetos a sus subclases para evitar acoplamiento con clases concretas.
- Solución: Define un método de creación en una clase base y permite que las subclases lo sobrescriban.
- Elementos:
 - Product (interfaz o clase abstracta).
 - ConcreteProduct.
 - Creator (define el factory method).
 - ConcreteCreator.
- Ventajas:
 - Aumenta la flexibilidad del código.
 - Elimina la dependencia directa de clases concretas.
 - Facilita la extensión del sistema.
- Desventajas:
 - Puede conducir a una gran cantidad de clases.
 - Complejidad extra comparado con una simple instancia directa.
- Aplicaciones:
 - Frameworks donde las bibliotecas llaman a código específico del usuario.
 - Cuando las clases no pueden anticipar qué clases de objetos deben crear.
- Gráficamente:



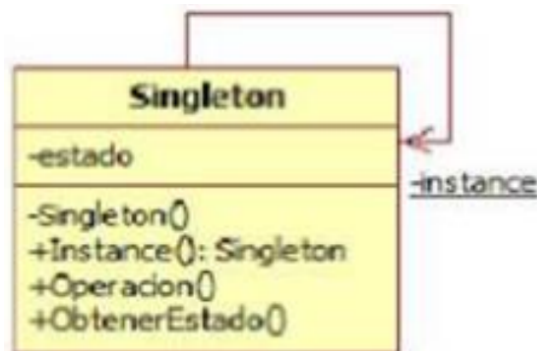
3. Prototipo (Prototype)

- Descripción: Permite crear nuevos objetos copiando una instancia existente (el prototipo) en lugar de crear nuevas instancias desde cero.
- Problema: Evitar la creación costosa o compleja de objetos desde cero.
- Solución: Se utiliza un objeto prototipo que se clona para generar nuevas instancias.
- Elementos:
 - Prototipo (interfaz con método clone()).
 - ConcretePrototype.
 - Cliente.
- Ventajas:
 - Reduce el costo de crear objetos complejos.
 - Útil para evitar la creación de subclasses solo para instanciación.
- Desventajas:
 - La clonación puede ser complicada si el objeto contiene referencias circulares o no copiables.
 - Puede haber problemas con la implementación del método de clonación.
- Aplicaciones:
 - Cuando el coste de creación es elevado (por ejemplo, objetos que requieren mucho tiempo para su configuración).
 - Juegos o editores gráficos donde se necesitan duplicar elementos rápidamente.
- Gráficamente:



5. Singleton (Singular)

- Descripción: Asegura que una clase tenga una única instancia y proporciona un punto de acceso global a ella.
- Problema: Se necesita exactamente una instancia de una clase y debe ser accesible globalmente.
- Solución: La propia clase controla su instancia y proporciona un método para obtenerla.
- Elementos:
 - Clase con constructor privado
 - Método estático para acceso global
 - Variable estática para almacenar la instancia
- Ventajas:
 - Controla el acceso único a recursos compartidos.
 - Ahorro de recursos (por ejemplo, conexión a BD).
- Desventajas:
 - Dificulta las pruebas (mocking).
 - Puede violar el principio de responsabilidad única.
 - Puede convertirse en una especie de variable global disfrazada.
- Aplicaciones:
 - Gestores de configuración.
 - Acceso a archivos de log.
 - Manejo centralizado de conexiones a BD.
- Gráficamente:

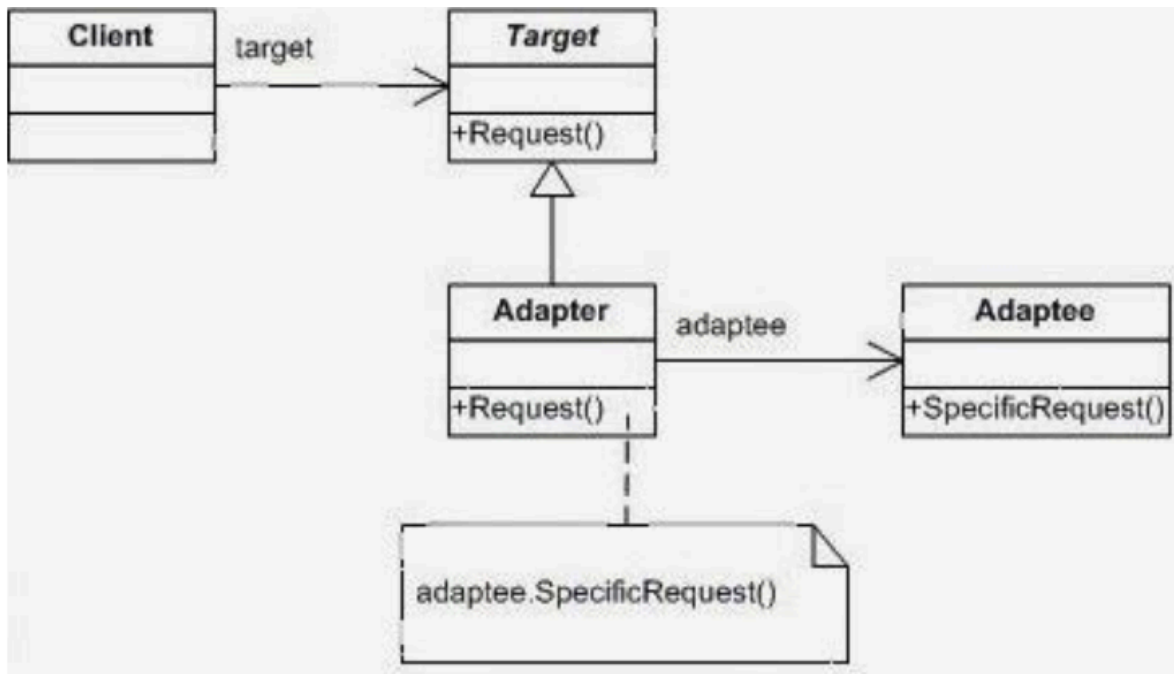


Patrones estructurales.

Se ocupan de la composición de clases y objetos para formar estructuras más grandes. Incluye a Adaptador, Puente, Compuesto, Decorador, Fachada, Peso Mosca, Apoderado entre otros.

1. Adaptador (Adapter)

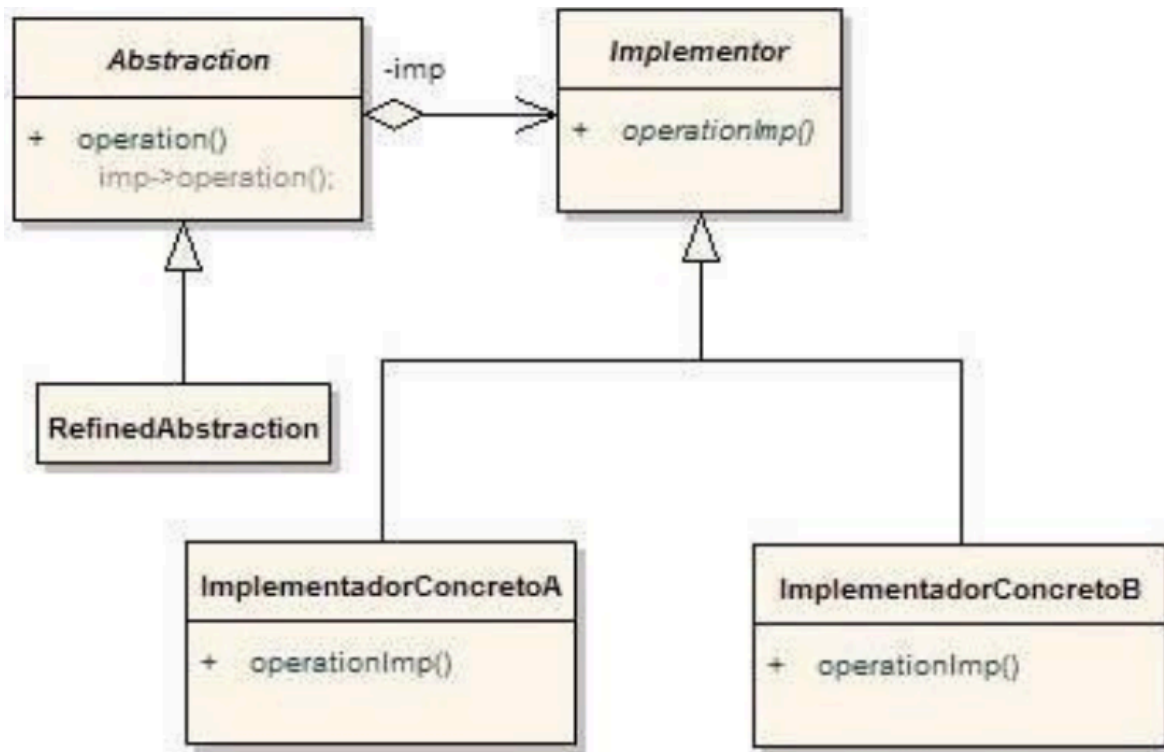
- Descripción: Convierte la interfaz de una clase en otra que el cliente espera. Permite que clases con interfaces incompatibles colaboren.
- Problema: Una clase necesita usar otra que tiene una interfaz incompatible.
- Solución: Introduce una clase adaptadora que convierte la interfaz de una clase en otra esperada por el cliente.
- Elementos:
 - Target (interfaz esperada).
 - Adaptee (clase existente).
 - Adapter (traduce la interfaz).
 - Cliente
- Ventajas:
 - Permite la reutilización de clases existentes sin modificar su código.
 - Favorece la compatibilidad entre interfaces diferentes.
- Desventajas:
 - Introduce complejidad adicional.
 - Puede ser una solución temporal mal usada para evitar rediseñar.
- Aplicaciones:
 - Integración de código heredado con nuevos sistemas.
 - Cuando se desea usar una clase sin modificarla pero adaptarla a otra interfaz.
- Gráficamente:



2. Puente (Bridge)

- Descripción: Separa una abstracción de su implementación, permitiendo que ambas evolucionen independientemente.
- Problema: Necesidad de variar tanto una abstracción como su implementación sin crear combinaciones múltiples de clases.
- Solución: Divide la jerarquía en dos: abstracción e implementación, usando composición.
- Elementos:
 - Abstraction.
 - RefinedAbstraction.
 - Implementor.
 - ConcreteImplementor.
- Ventajas:
 - Alta flexibilidad para cambiar la implementación sin afectar al cliente.
 - Favorece el desacoplamiento y el crecimiento del sistema.
- Desventajas:
 - Mayor número de clases.
 - Puede ser innecesario para casos simples.

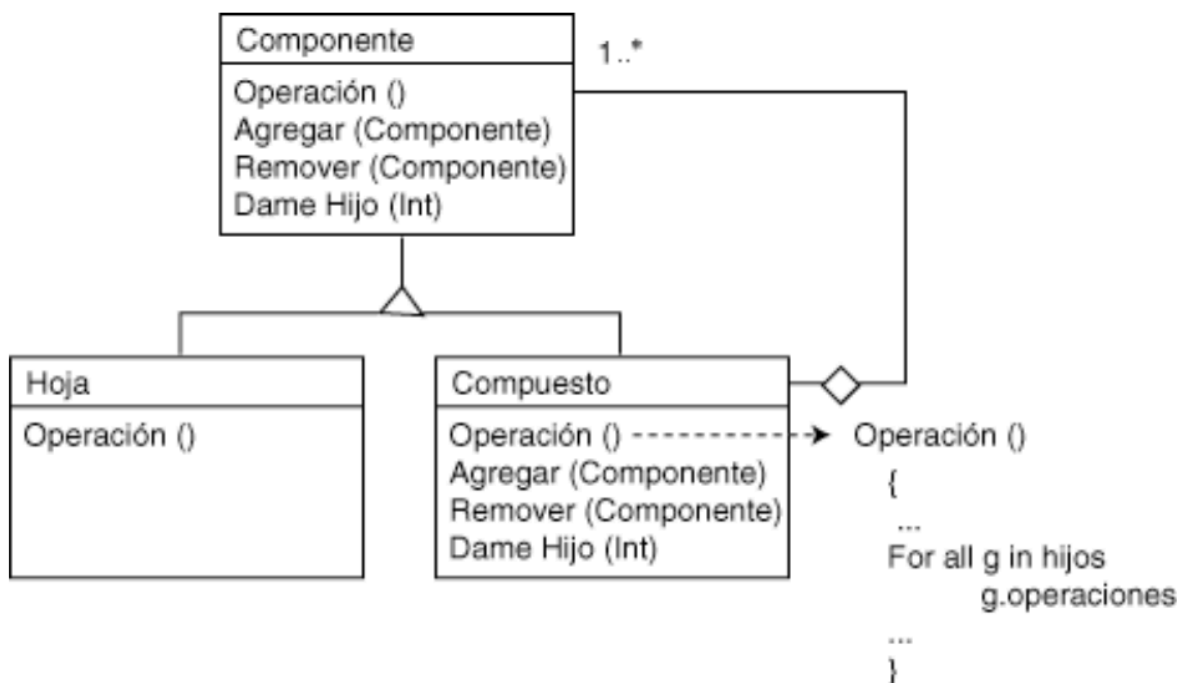
- Aplicaciones:
 - Interfaces gráficas que se implementan en múltiples plataformas.
 - Dispositivos que se pueden controlar mediante varias tecnologías.
- Gráficamente:



3. Compuesto (Composite)

- Descripción: Compone objetos en estructuras de árbol para representar jerarquías parte-todo. Permite tratar objetos individuales y compuestos uniformemente.
- Problema: Necesidad de manipular objetos individuales y grupos de objetos de forma similar.
- Solución: Define una interfaz común para todos los objetos y permite que los compuestos contengan otros objetos.
- Elementos:
 - Component (interfaz).
 - Leaf (objeto individual).
 - Composite (objeto compuesto).

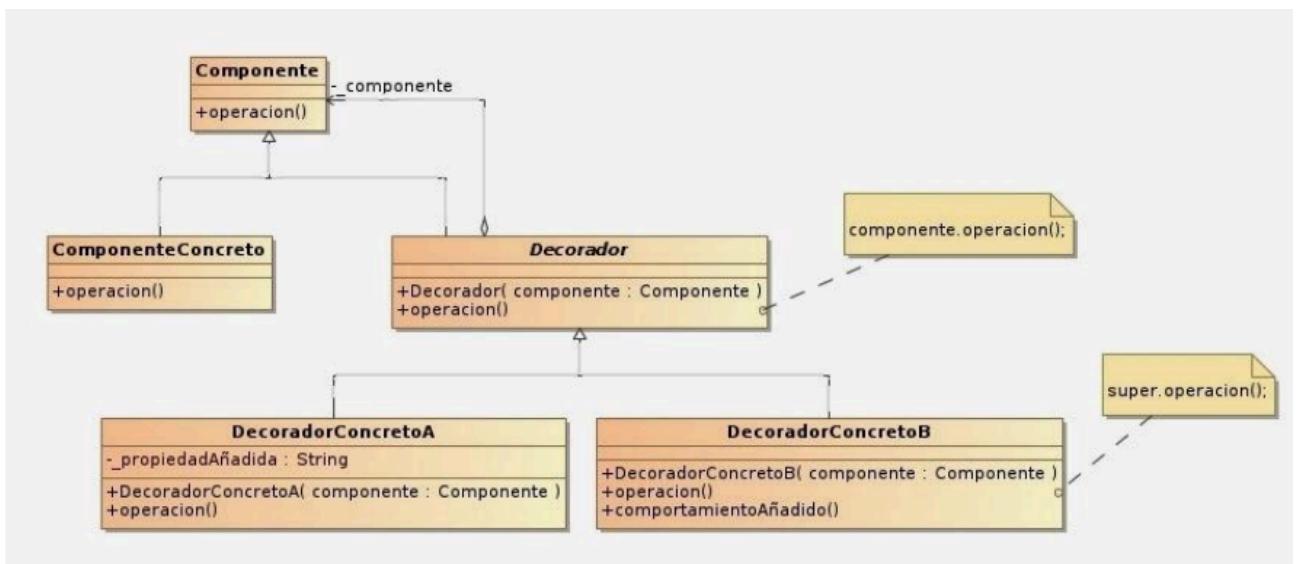
- Cliente.
- Ventajas:
 - Simplifica al cliente al tratar objetos individuales y compuestos por igual.
 - Estructura jerárquica clara y extensible.
- Desventajas:
 - Puede ser complejo implementar operaciones si los componentes difieren mucho.
 - Aumenta el número de clases.
- Aplicaciones:
 - Interfaces gráficas (componentes visuales dentro de contenedores).
 - Sistemas jerárquicos (por ejemplo, archivos y carpetas).
- Gráficamente:



4. Decorador (Decorator)

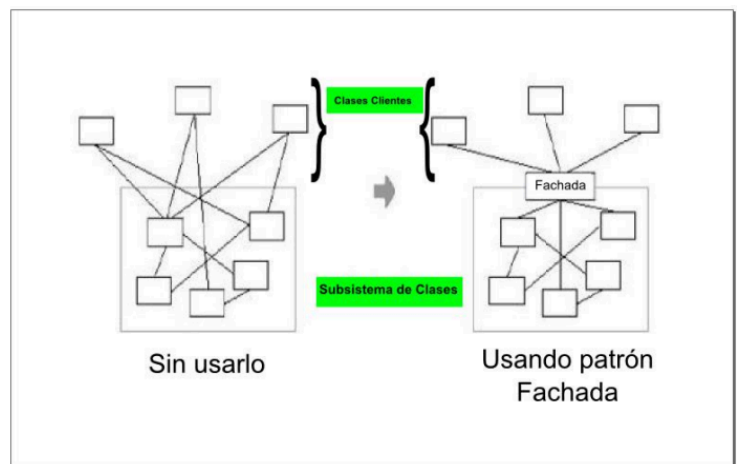
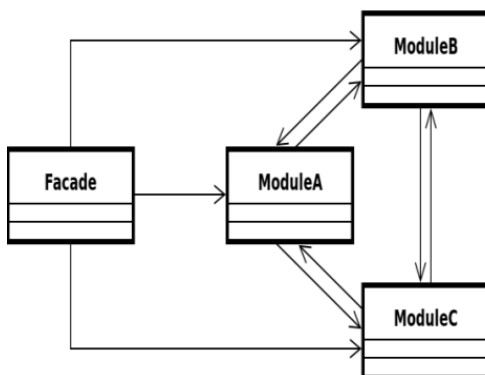
- Descripción: Añade responsabilidades a un objeto dinámicamente. Es una alternativa flexible a la herencia para extender funcionalidades.
- Problema: Necesidad de añadir funcionalidades a objetos individuales sin afectar a toda la clase.

- Solución: Crea clases decoradoras que envuelvan al objeto original, extendiendo su comportamiento.
- Elementos:
 - Component.
 - ConcreteComponent.
 - Decorator.
 - ConcreteDecorator.
- Ventajas:
 - Composición flexible de funcionalidades.
 - Favorece la adherencia al principio de abierto/cerrado.
- Desventajas:
 - Puede volverse difícil de entender con muchos decoradores anidados.
 - No controla la ordenación de decoradores (afecta comportamiento final).
- Aplicaciones:
 - Sistemas donde se necesita extender funcionalidades en tiempo de ejecución.
 - Interfaces de usuario con estilos dinámicos.
- Gráficamente:



5. Fachada (Facade)

- Descripción: Proporciona una interfaz unificada a un conjunto de interfaces en un subsistema. Simplifica el uso del sistema para el cliente.
- Problema: Sistemas complejos que exponen muchas clases y funcionalidades dificultan su uso.
- Solución: Crear una clase fachada que encapsule la complejidad del subsistema y ofrezca métodos sencillos.
- Elementos:
 - Fachada.
 - Subsistemas.
 - Cliente.
- Ventajas:
 - Reduce la complejidad de acceso a subsistemas.
 - Promueve el desacoplamiento.
- Desventajas:
 - Puede ocultar funcionalidades útiles del subsistema.
 - Introduce una capa adicional.
- Aplicaciones:
 - API públicas para bibliotecas complejas.
 - Integración con sistemas legados.
- Gráficamente:



Discusión (Patrones estructurales).

Comparación de patrones como Adaptador, Bridge, Composite, y Decorador, destacando sus similitudes estructurales y diferencias en propósito y aplicación.

La discusión enfatiza cómo estos patrones estructurales facilitan la composición y la flexibilidad del sistema, permitiendo un diseño más modular y adaptable a cambios futuros.

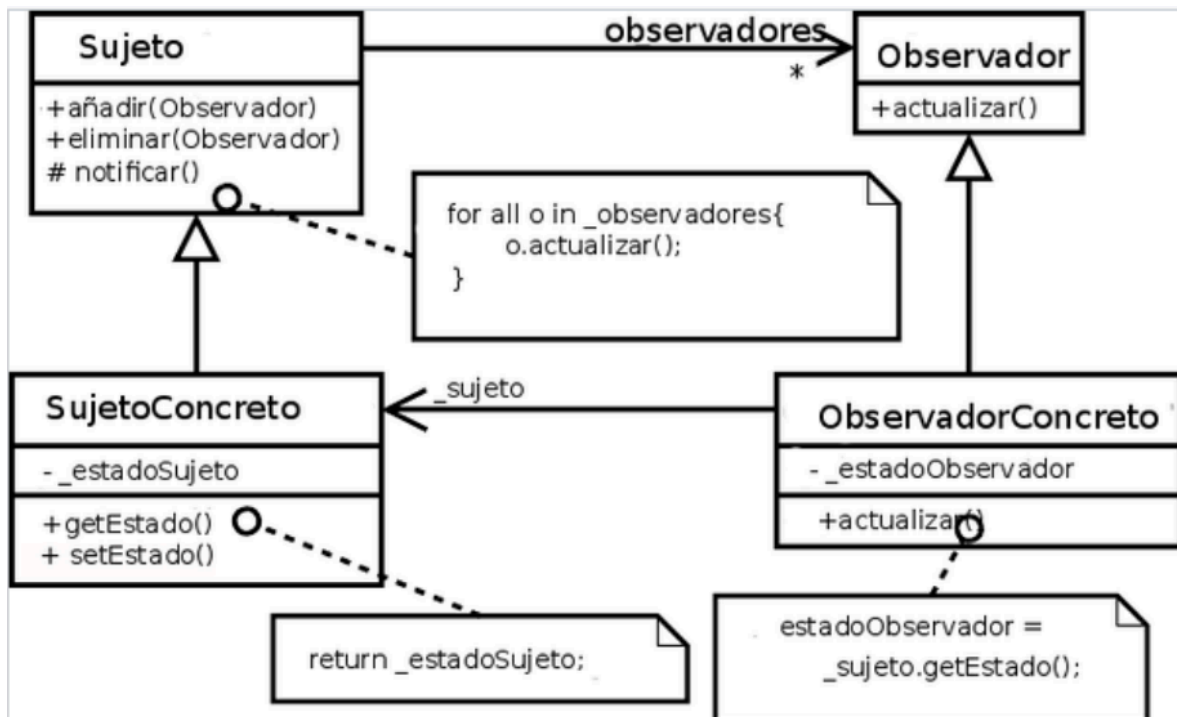
Patrones de comportamiento.

Se enfocan en algoritmos y la asignación de responsabilidades entre objetos. Incluyen Método Cadena de Responsabilidad, Comando, Iterador, Mediador, Memento, Observador, Estado, Estrategia, Visitante, Método Plantilla, entre otros.

1. Observador (Observer)

- Descripción: Define una dependencia de uno-a-muchos entre objetos, de manera que cuando uno cambia su estado, todos sus dependientes son notificados.
- Problema: Mantener objetos sincronizados sin acoplamiento fuerte.
- Solución: Los objetos observadores se registran y son notificados de cambios.
- Elementos:
 - Subject.
 - Observer.
 - ConcreteSubject.
 - ConcreteObserver.
- Ventajas:
 - Favorece el acoplamiento bajo.
 - Facilita el flujo reactivo de datos.
- Desventajas:
 - Difícil de depurar si hay muchos observadores.
 - Riesgo de referencias circulares.
- Aplicaciones:
 - Interfaces de usuario.

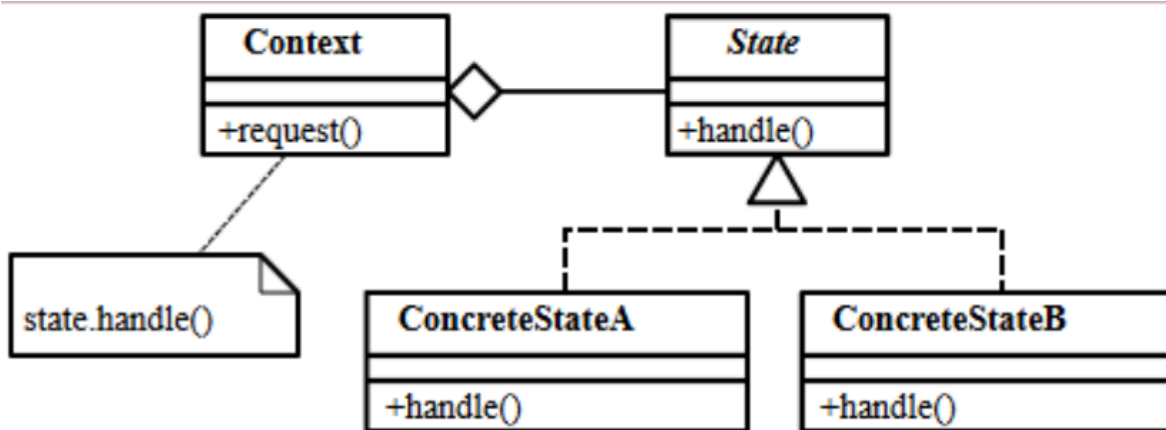
- Sistemas de eventos.
- Gráficamente:



2. Estado (State)

- Descripción: Permite que un objeto altere su comportamiento cuando cambia su estado interno.
- Problema: Un objeto cambia su comportamiento dependiendo de su estado.
- Solución: Representar cada estado como una clase y delegar comportamiento.
- Elementos:
 - Context.
 - State.
 - ConcreteState.
- Ventajas:
 - Organiza el comportamiento basado en estados.
 - Evita múltiples sentencias condicionales.
- Desventajas:
 - Aumenta el número de clases.

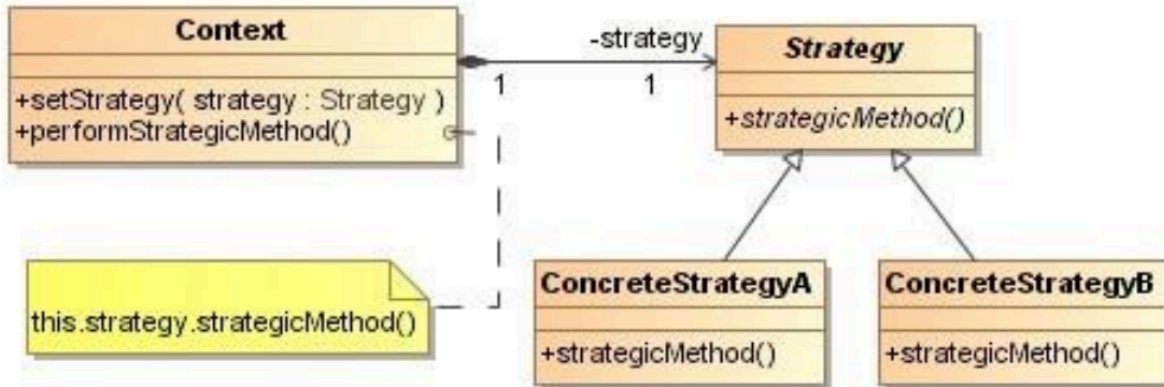
- Puede complicar el diseño.
- Aplicaciones:
 - Autómatas finitos.
 - Ciclos de vida de objetos.
- Gráficamente:



3. Estrategia (Strategy)

- Descripción: Define una familia de algoritmos, encapsula cada uno, y los hace intercambiables.
- Problema: Variaciones de comportamiento que deben cambiarse en tiempo de ejecución.
- Solución: Encapsular cada algoritmo como un objeto independiente.
- Elementos:
 - Strategy.
 - ConcreteStrategy.
 - Context.
- Ventajas:
 - Elimina estructuras condicionales.
 - Facilita el intercambio de comportamientos.
- Desventajas:
 - Aumenta el número de clases.
 - El cliente debe conocer las estrategias.
- Aplicaciones:

- Algoritmos de ordenamiento, compresión, encriptación.
- Políticas de negocio.
- Gráficamente:



Discusión (Patrones de comportamiento).

Comparación entre Método Plantilla y Estrategia, destacando que el primero utiliza herencia para variar parte de un algoritmo mientras que el segundo utiliza delegación para variar todo el algoritmo.

El futuro de los patrones.

Aunque actualmente hay un número relativamente pequeño de patrones, se espera que su identificación y catalogación continúe, facilitando que la ingeniería de software se parezca más a otras disciplinas de ingeniería. Eventualmente, los patrones y componentes se volverán análogos a pequeños circuitos electrónicos.

Tema 6. Implementación y Evolución.

Técnicas de transformación del diseño al código. Creación de las definiciones de las clases a partir de los DCDs. Creación de métodos a partir de los diagramas de interacción. Trazabilidad como requerimiento de calidad. Dinámica de evolución de los programas. Mantenimiento del software. Procesos de evolución. Evolución de sistemas heredados.

Técnicas de Transformación del Diseño al Código.

Es el proceso de convertir el diseño en código ejecutable. Los artefactos de diseño como los Diagramas de Clases de Diseño (DCDs) son utilizados como entrada para generar el código fuente, bases de datos, páginas web y scripts de prueba. La programación implica elegir un lenguaje de programación y traducir modelos visuales a implementación. Se subraya la importancia de mantener la trazabilidad entre el diseño y el código, lo cual puede ser facilitado por herramientas CASE y la ingeniería en reverso.

Dinámica de Evolución de los Programas.

Después de que un sistema es desarrollado, inevitablemente sufre cambios para mantenerse útil. Nuevos requerimientos y cambios en los existentes surgen a medida que el software es utilizado. El desarrollo del software, por lo tanto, no se detiene con la entrega del sistema, sino que continúa durante su vida útil. Las organizaciones invierten significativamente en la evolución de sus sistemas para mantener el valor de estos activos.

Mantenimiento del Software.

El mantenimiento del software implica modificar un sistema software después de su entrega para corregir fallos, mejorar su rendimiento u otros atributos, o adaptarlo a un entorno cambiado. Es una fase crucial debido a la dependencia de las organizaciones en sus sistemas software y la inversión significativa realizada en estos sistemas.

Predicción del Mantenimiento.

La predicción del mantenimiento se enfoca en anticipar las necesidades futuras de mantenimiento para planificar adecuadamente los recursos y esfuerzos necesarios. Incluye estimaciones sobre la frecuencia y el tipo de mantenimiento que un sistema podría requerir basado en su historial y el análisis de tendencias.

Procesos de Evolución.

Los procesos de evolución detallan los métodos y prácticas utilizadas para manejar los cambios en los sistemas software durante su ciclo de vida. Esto incluye estrategias para implementar nuevas funcionalidades, corregir errores, y adaptar el sistema a nuevos entornos o requisitos.

Reingeniería de Sistemas.

La reingeniería de sistemas implica examinar y modificar un sistema existente para mejorar su calidad y rendimiento. Esto puede incluir la reestructuración del código, la actualización de la tecnología utilizada, y la mejora de la documentación. La reingeniería busca prolongar la vida útil del sistema y mejorar su mantenibilidad.

Evolución de Sistemas Heredados.

Los sistemas heredados son sistemas antiguos que todavía están en uso. Su evolución es crucial para mantener su relevancia y operatividad. Este subtema aborda las estrategias para modernizar estos sistemas sin interrumpir su funcionamiento, incluyendo la integración de nuevas tecnologías y la migración de datos y funcionalidades a plataformas más modernas.

Tema 8. Reingeniería.

Conceptos fundamentales. La reingeniería de los procesos del negocio. Principios de reingeniería de procesos de Negocio. Reingeniería del Software. Mantenimiento. Un modelo de proceso de Reingeniería del software. Ingeniería Inversa. Ingeniería Inversa desde la Interfaz de usuario. Herramientas Case e Ingeniería Inversa.

Conceptos fundamentales:

La reingeniería es esencial para las compañías con software obsoleto que necesitan adaptarse a nuevas reglas de negocio para ser más efectivas y competitivas. Esto puede implicar la creación de nuevos sistemas o la modificación de aplicaciones existentes.

Características Básicas:

- ¿Qué es?: Se refiere a productos tecnológicos que han envejecido, se rompen con frecuencia y requieren reconstrucción en lugar de reemplazo. La reingeniería mejora funcionalidad, rendimiento, fiabilidad y mantenimiento.
- ¿Quién lo hace?: A nivel de negocio, especialistas en negocio o consultoras. A nivel de software, ingenieros de software.
- ¿Por qué es importante?: Debido a los constantes cambios en las demandas de funciones de negocios y tecnología, es crucial rediseñar tanto los negocios como el software que los soporta para mantenerse competitivos.
- ¿Cuáles son los pasos?: Define metas comerciales, evalúa procesos de negocios existentes, y crea procesos revisados. En software, implica análisis de inventarios, estructuración de documentos, ingeniería inversa y reestructuración de datos.

Reingeniería de procesos de negocio:

La reingeniería de procesos de negocio (RPN) busca implementar cambios radicales en los procesos para lograr avances significativos. Un proceso de

negocio se define como un conjunto de tareas lógicamente relacionadas para obtener un resultado específico.

Un modelo de procesos de reingeniería del Software:

Las aplicaciones que han estado en uso por años pueden volverse inestables y difíciles de mantener debido a múltiples modificaciones y mejoras realizadas sin seguir buenas prácticas de ingeniería de software. La reingeniería del software ayuda a estabilizar y mejorar estas aplicaciones.

Mantenimiento:

El mantenimiento del software abarca más que solo corregir errores, incluyendo adaptaciones a cambios externos, mejoras solicitadas por usuarios y reingeniería de aplicaciones. Este proceso consume un 80% de los esfuerzos de mantenimiento

Ingeniería Inversa:

La ingeniería inversa implica derivar representaciones de diseño de procedimientos, estructuras de datos y programas, y modelos de flujo de datos y control. A medida que crece el nivel de abstracción, se proporciona más información al ingeniero del software, facilitando la comprensión de los programas.

Ingeniería Inversa desde la Interfaz de usuario:

La ingeniería inversa de una interfaz de usuario requiere especificar su estructura y comportamiento. Se debe entender las acciones básicas procesadas por la interfaz y la respuesta del sistema a estas acciones para poder reconstruirla eficazmente.

Tema 9. Gestión de Cambios: Cambios - Configuración - Riesgos.

Definiciones. Definir el Plan de Cambios. Problemática de la gestión de Cambios. La importancia de la Gestión de Configuración. Herramientas de soporte a la Gestión de Configuración. Auditorías de la Configuración. Gestión de la Liberación. Conceptos de la Liberación. Seguimiento de la liberación y Control de Conformidad del Usuario. Identificación de los Riesgos. Generación del plan. Estrategias del Riesgo Proactivas Vs Reactivas. Enfoques. Evaluación de Impacto. Supervisión y Gestión de Riesgos. Normas relacionadas.

Gestión de Cambios.

- **Objetivo:** Asegurar que todos los cambios en la infraestructura y servicios TI se realicen e implementen adecuadamente, siguiendo procedimientos estándar.
- **Justificación y Calidad:** Los cambios deben estar justificados, no perjudicar la calidad del servicio TI, y deben ser registrados, clasificados, y documentados correctamente.
- **Pruebas y Reversión:** Los cambios deben ser testeados en un entorno de prueba y deben tener planes de reversión en caso de problemas.
- **CMDB:** Los cambios deben reflejarse en la CMDB (Base de Datos de Gestión de la Configuración).
- **Planes de Reversión:** Deben existir planes para revertir los cambios en caso de problemas tras la implementación.

CMDB.

- **CMDB (Base de Datos de Gestión de la Configuración):** Busca consolidar los datos de múltiples fuentes de TI en un modelo de datos común, creando una única fuente confiable. Esto facilita una mejor gestión de los servicios empresariales, asegurando que todos los servicios utilicen los mismos datos de configuración.

- **Errores al Conceptualizar una CMDB:**

1. No es un simple inventario: Incluye las relaciones entre los activos.
2. No es una migración de datos a una fuente única: Mejora el control y gestión de las herramientas existentes sin unificarlas todas.

- **Metas de una CMDB:**

- Contabilizar activos y configuraciones de TI.
- Proveer información confiable sobre estos activos y configuraciones.

- **Beneficios de una CMDB:** Proporciona una mejor gestión y control sobre los activos de TI y sus relaciones, facilitando la toma de decisiones y mejorando la eficiencia operativa.

- **CMDB como Mecanismo de Control:** Permite controlar las versiones y estados de los elementos de configuración (CIs), asegurando la integridad y exactitud de los datos de configuración.

- **Las CMDB como Mecanismo de Integración:** Facilita la integración de diferentes sistemas y herramientas de TI, mejorando la coherencia y la coordinación entre ellos.

- **Las CMDB como Mecanismo de Soporte para la Toma de Decisiones:** Provee información precisa y actualizada que es crucial para la toma de decisiones informadas en la gestión de servicios de TI.

- **Los Datos son la Clave de Todo:** La precisión y fiabilidad de los datos en una CMDB son esenciales para su efectividad como herramienta de gestión y control.

- **Relaciones entre las CIs:** La CMDB debe incluir las relaciones entre los diferentes elementos de configuración, proporcionando una visión integral y detallada de la infraestructura de TI.

Gestión de Cambios. Actividades Principales.

- **Principales Beneficios:** Mejora la estabilidad y la calidad de los servicios de TI, asegurando que los cambios se implementen de manera controlada y efectiva.
- **Principales Inconvenientes de Implementación:** Los procedimientos excesivamente restrictivos pueden dificultar la mejora, mientras que una trivialización del proceso de cambio puede provocar una falta de estabilidad.

Gestión de la Liberación (Release).

- **Definiciones:**
 - **Release:** Conjunto de cambios autorizados a un servicio TI.
 - **Biblioteca de Software Definitiva (DSL):** Almacena y protege las versiones definitivas autorizadas de todos los CIs de software.
 - **Almacén de Hardware Definitivo (DHS):** Mantiene componentes y módulos de repuestos al mismo nivel que los sistemas de producción.
 - **Versión:** Instancia identificada de un CI para seguimiento y auditoría de cambios.
 - **Política de Release:** Documento que define roles, responsabilidades y directrices para la gestión de releases.
 - **Unidad de Release:** Nivel de granularidad más apropiado para cada CI.
- **Tipos de Release:**
 - **Release Delta:** Incluye solo los CIs que han cambiado o son nuevos.
 - **Release Completa:** Todos los componentes se construyen, prueban, distribuyen o implementan en conjunto.

- **Paquete de Release:** Incluye al menos dos releases, ofreciendo estabilidad y reduciendo la frecuencia de cambios.
- **Release de Emergencia:** Corrige una pequeña cantidad de problemas conocidos.

Gestión de la Configuración.

- **Objetivo:** Proporcionar un modelo lógico de la infraestructura o del servicio mediante la identificación, control, mantenimiento y verificación de las versiones de los elementos de configuración (CIs) existentes.
- **Definiciones:**
 - **Infraestructura:** Incluye hardware, software y documentación.
 - **Elemento de Configuración (CI):** Componente de la infraestructura bajo control de la Gestión de la Configuración.
 - **Atributos:** Detalles que identifican a los CIs (ubicación, número de serie, versión, propietarios, etc.).
 - **Relaciones:** Descripción de las interfaces entre los CIs.
 - **CMDB:** Base de datos que contiene un registro de todos los CIs asociados con la infraestructura de TI.
- **Actividades:**
 - Planeación de la Gestión de Configuraciones: Definir el contexto y planificar la gestión.
 - Identificación: Determinar el alcance y nivel de detalle de los CIs.
 - Control: Registrar, archivar y actualizar la información.

- Estado del Ciclo de Vida: Mantener el historial y la ruta de auditoría.
- Verificación y Auditoría: Identificar desviaciones y asegurar la precisión de los datos.

Análisis de Riesgo.

- **Riesgo:** Proceso de identificar sistemáticamente los factores de riesgo y evaluar sus propiedades.
- **¿Cómo Medimos el Riesgo?:**
 - **Probabilidad:** Likelihood de que ocurra la amenaza.
 - **Impacto:** Consecuencias que podría ocasionar la amenaza.
- **Métodos de Evaluación del Riesgo:** Herramientas y técnicas para ordenar la toma de decisiones basada en la evaluación del riesgo.

ITIL

- **Los Objetivos Fueron:** Mejorar la eficiencia y efectividad de la gestión de servicios de TI mediante la implementación de buenas prácticas.
- **Conceptos:** Framework de mejores prácticas para la gestión de servicios de TI.
- **ITIL y Otros Estándares:**
 - **Comparación con Otros Estándares:**
 - ISO/IEC 20000: ITIL proporciona las mejores prácticas, mientras que ISO/IEC 20000 establece un estándar que las organizaciones pueden certificar.
 - CMMI: CMMI se centra en la mejora de procesos a nivel organizacional, mientras que ITIL se centra específicamente en la gestión de servicios de TI.
 - PMBOK: Se enfoca en la gestión de proyectos en general, mientras que ITIL se enfoca en la gestión de servicios de TI.

- Cobit: Se enfoca en la gobernanza de TI y el cumplimiento normativo, mientras que ITIL se centra en la gestión de servicios de TI.

– **ITIL y su Relación con Otros Estándares:**

- Complementariedad: ITIL puede ser utilizado junto con otros estándares y marcos para proporcionar una gestión de servicios de TI más completa y efectiva.
- Integración: Las mejores prácticas de ITIL pueden ser integradas con los enfoques y estándares mencionados para mejorar la gestión global de TI en una organización.

● **Beneficios de ITIL:**

- Mejora la calidad de los servicios de TI.
- Aumenta la eficiencia operativa.
- Facilita la alineación de TI con los objetivos de negocio.
- **ISO/IEC 20000:** Estándar internacional de gestión de servicios de TI basado en las mejores prácticas de ITIL.
- **Competisof:** Mejora de Procesos para Fomentar la Competitividad de la Pequeña y Mediana Industria del Software de Iberoamérica
 - Alcance: Definir el ámbito de aplicación del modelo.
 - Criterios Empleados para la Elaboración de los Modelos: Factores considerados en el desarrollo del modelo.